

Neo4j และ Graph Database

ระบบแนะนำหนังสือ และระบบแนะนำชมรม ด้วย Cypher Query

จัดทำโดย

นายจิรภัทร จันทร์มล

รหัสนักศึกษา 664245026 หมู่เรียน 66/43

มหาวิทยาลัยราชภัฏนครปฐม

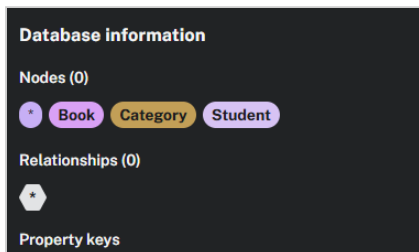
ส่วนที่ 1: ระบบแนะนำหนังสือ (School Book Recommendation)

โรงเรียนต้องการเก็บข้อมูลนักเรียน หนังสือ หมวดหนังสือ ความเป็นเพื่อน และประวัติการยืม เพื่อค้นหาข้อมูลและแนะนำหนังสือจากเพื่อน โดยใช้ Graph Database

1.1 สร้าง Constraint ของตัวระบุ

กำหนดให้รหัสของ Student, Book และ Category ต้องไม่ซ้ำกัน

```
CREATE CONSTRAINT student_id_unique IF NOT EXISTS
FOR (s:Student) REQUIRE s.student_id IS UNIQUE;
CREATE CONSTRAINT book_id_unique IF NOT EXISTS
FOR (b:Book) REQUIRE b.book_id IS UNIQUE;
CREATE CONSTRAINT category_id_unique IF NOT EXISTS
FOR (c:Category) REQUIRE c.category_id IS UNIQUE;
```



1.2 สร้างข้อมูลตัวอย่างและดูกราฟทั้งหมด

สร้างนักเรียน 4 คน หนังสือ 5 เล่ม หมวดหนังสือ 2 หมวด พร้อมความสัมพันธ์ FRIEND_OF, BORROWED และ IN_CATEGORY รวม 11 nodes และ 16 relationships

```
CREATE
  (anan:Student {student_id:'S001', name:'Anan', grade:'M.4'}),
  (mali:Student {student_id:'S002', name:'Mali', grade:'M.4'}),
  (somchai:Student {student_id:'S003', name:'Somchai', grade:'M.5'}),
  (nida:Student {student_id:'S004', name:'Nida', grade:'M.4'}),
  (comp:Category {category_id:'C01', name:'Computer'}),
  (db:Category {category_id:'C02', name:'Database'}),
  (b1:Book {book_id:'B101', title:'Python for Beginners'}),
  (b2:Book {book_id:'B102', title:'Artificial Intelligence Basics'}),
  (b3:Book {book_id:'B103', title:'Data Science for Students'}),
  (b4:Book {book_id:'B104', title:'Graph Thinking'}),
  (b5:Book {book_id:'B105', title:'Database Design 101'}),
  (b1)-[:IN_CATEGORY]->(comp), (b2)-[:IN_CATEGORY]->(comp),
  (b3)-[:IN_CATEGORY]->(comp), (b4)-[:IN_CATEGORY]->(comp),
  (b5)-[:IN_CATEGORY]->(db),
  (anan)-[:FRIEND_OF]->(mali), (anan)-[:FRIEND_OF]->(somchai),
  (anan)-[:FRIEND_OF]->(nida), (mali)-[:FRIEND_OF]->(somchai),
  (anan)-[:BORROWED {borrow_date: date('2026-09-01')}]>(b1),
  (mali)-[:BORROWED {borrow_date: date('2026-09-02')}]>(b2),
  (mali)-[:BORROWED {borrow_date: date('2026-09-03')}]>(b4),
  (somchai)-[:BORROWED {borrow_date: date('2026-09-04')}]>(b3),
  (somchai)-[:BORROWED {borrow_date: date('2026-09-05')}]>(b4),
  (nida)-[:BORROWED {borrow_date: date('2026-09-06')}]>(b2),
  (nida)-[:BORROWED {borrow_date: date('2026-09-07')}]>(b5);

MATCH p = ()-[]->() RETURN p;
```

Database information

Nodes (11)

Book Category Student

Relationships (16)

BORROWED FRIEND_OF IN_CATEGORY

6945006f\$ MATCH p = ()-[]->() RETURN p;

Graph Table Raw

Search graph...

Results overview

Nodes (11)

* (11) Book (5) Category (2) Student (4)

Relationships (16)

* (16) BORROWED (7) FRIEND_OF (4) IN_CATEGORY (5)

Started streaming 16 records after 14 ms and completed after 15 ms.

1.3 คำถามที่ 1: นักเรียน S001 เคยยืมหนังสืออะไร?

ใช้ MATCH + WHERE กรองนักเรียนรหัส S001 แล้วดึงชื่อหนังสือและวันที่ยืมจาก relationship BORROWED

```
MATCH (s:Student)-[r:BORROWED]->(b:Book)
WHERE s.student_id = 'S001'
RETURN s.name AS student, b.title AS book, r.borrow_date AS borrowed_on;
```

6945006f\$ MATCH (s:Student)-[r:BORROWED]->(b:Book) WHERE s.student_id = 'S001' RETURN s.name AS student, b.title AS book, r.borrow ...

Table Raw

Search table...

Graph chip

student	book	borrowed_on
"Anan"	"Python for Beginners"	2026-09-01

Started streaming 1 record after 24 ms and completed after 25 ms.

ผลลัพธ์: Anan (S001) ยืมหนังสือ Python for Beginners เมื่อ 2026-09-01

1.4 คำถามที่ 2 (1 Hop): ใครเป็นเพื่อนของ S001?

เดิน 1 hop ผ่าน FRIEND_OF โดยไม่ระบุทิศทาง เพราะความเป็นเพื่อนเป็นความสัมพันธ์สองทาง

```
MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
RETURN friend.student_id, friend.name;
```



6945006f\$ MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student) RETURN friend.student_id, friend.name; ... Run

Table	Raw
friend.student_id	friend.name
1 "S002"	"Mali"
2 "S003"	"Somchai"
3 "S004"	"Nida"

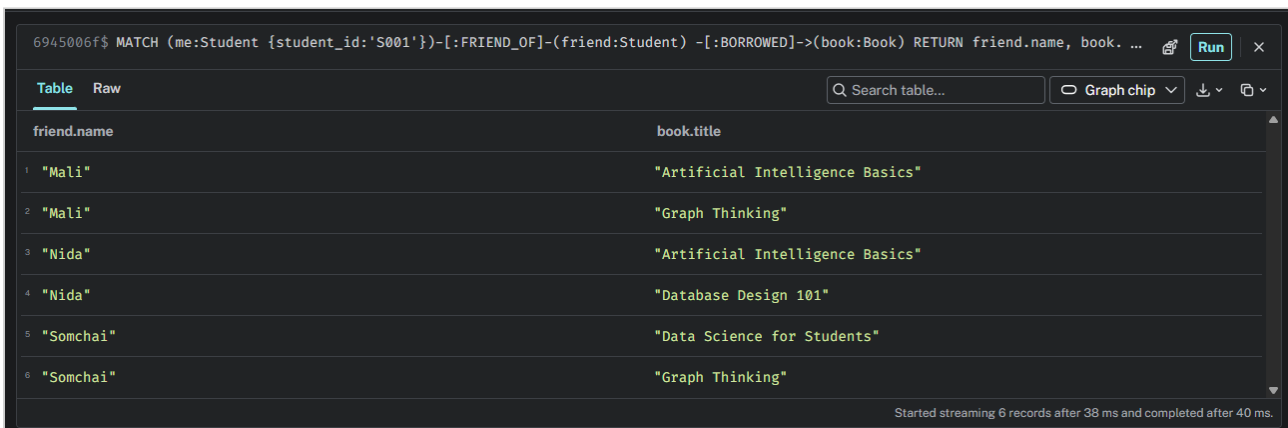
Started streaming 3 records after 23 ms and completed after 24 ms.

ผลลัพธ์: เพื่อนของ S001 มี 3 คน คือ Mali, Somchai และ Nida

1.5 คำถามที่ 3 (2 Hop): เพื่อนของ S001 เคยยืมหนังสืออะไร?

เดินต่อจากเพื่อน (hop ที่ 1) ผ่าน BORROWED ไปยังหนังสือ (hop ที่ 2)

```
MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
      -[:BORROWED]->(book:Book)
RETURN friend.name, book.title
ORDER BY friend.name;
```



6945006f\$ MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student) -[:BORROWED]->(book:Book) RETURN friend.name, book. ... Run

Table	Raw
friend.name	book.title
1 "Mali"	"Artificial Intelligence Basics"
2 "Mali"	"Graph Thinking"
3 "Nida"	"Artificial Intelligence Basics"
4 "Nida"	"Database Design 101"
5 "Somchai"	"Data Science for Students"
6 "Somchai"	"Graph Thinking"

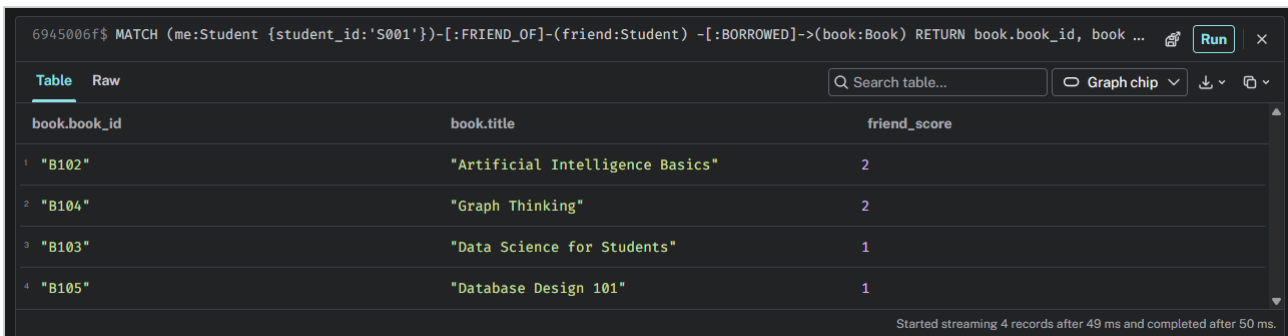
Started streaming 6 records after 38 ms and completed after 40 ms.

ผลลัพธ์: ได้ 6 รายการ จากเพื่อน 3 คน

1.6 คำถามที่ 4: หนังสือเล่มใดถูกยืมโดยเพื่อนหลายคน?

ใช้ `count(DISTINCT friend)` นับจำนวนเพื่อนที่ยืมหนังสือแต่ละเล่ม เป็นคะแนน `friend_score`

```
MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
      -[:BORROWED]->(book:Book)
RETURN book.book_id, book.title, count(DISTINCT friend) AS friend_score
ORDER BY friend_score DESC;
```



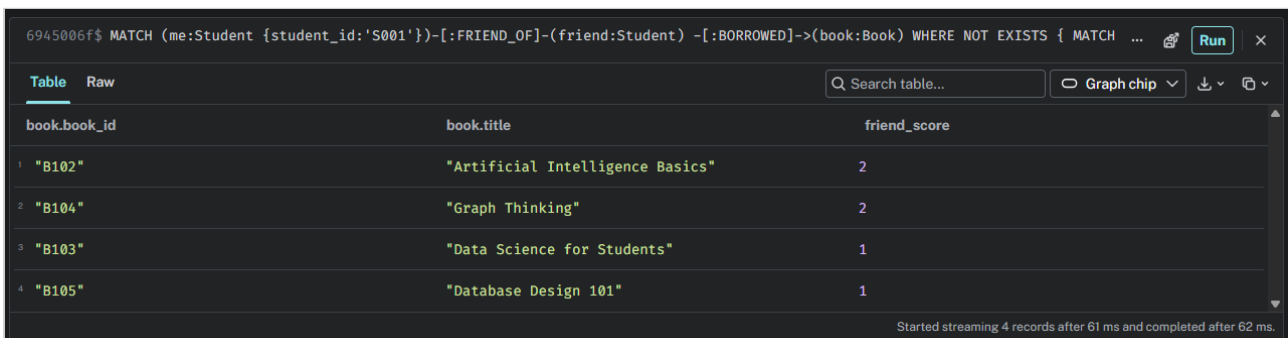
book.book_id	book.title	friend_score
"B102"	"Artificial Intelligence Basics"	2
"B104"	"Graph Thinking"	2
"B103"	"Data Science for Students"	1
"B105"	"Database Design 101"	1

ผลลัพธ์: Artificial Intelligence Basics และ Graph Thinking ถูกยืมโดยเพื่อน 2 คน ได้คะแนนสูงสุด

1.7 คำถามที่ 5: หนังสือใดควรเป็น Candidate แนะนำให้ S001

เพิ่มเงื่อนไข `WHERE NOT EXISTS` เพื่อตัดหนังสือที่ S001 เคยยืมแล้วออก

```
MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
      -[:BORROWED]->(book:Book)
WHERE NOT EXISTS { MATCH (me)-[:BORROWED]->(book) }
RETURN book.book_id, book.title, count(DISTINCT friend) AS friend_score
ORDER BY friend_score DESC, book.title;
```



book.book_id	book.title	friend_score
"B102"	"Artificial Intelligence Basics"	2
"B104"	"Graph Thinking"	2
"B103"	"Data Science for Students"	1
"B105"	"Database Design 101"	1

ผลลัพธ์เหมือนข้อ 1.6 เพราะหนังสือเล่มเดียวที่ S001 ยืมคือ Python for Beginners ซึ่งไม่มีเพื่อนคนใดยืม จึงไม่มีเล่มใดถูกตัดออก แต่หากในอนาคต S001 ยืมหนังสือที่เพื่อนเคยยืม เงื่อนไขนี้จะตัดเล่มนั้นออกจากรายการแนะนำโดยอัตโนมัติ

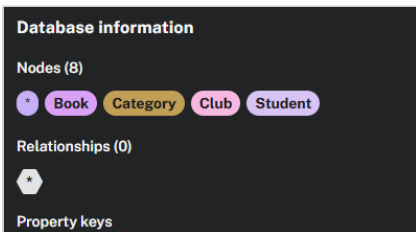
ส่วนที่ 2: ระบบแนะนำชมรม (School Club Recommendation)

ล้างข้อมูลส่วนที่ 1 ด้วย MATCH (n) DETACH DELETE n; แล้วสร้างข้อมูลชุดใหม่สำหรับระบบแนะนำชมรม

2.1 สร้าง Student 5 คน และ Club 3 ชมรม

สร้างนักเรียน 5 คน และชมรม 3 ชมรม ได้แก่ Coding Club, Robotics Club และ Music Club

```
CREATE
(:Student {student_id:'S001', name:'Anan'}),
(:Student {student_id:'S002', name:'Mali'}),
(:Student {student_id:'S003', name:'Somchai'}),
(:Student {student_id:'S004', name:'Nida'}),
(:Student {student_id:'S005', name:'Pim'}),
(:Club {club_id:'CL01', name:'Coding Club'}),
(:Club {club_id:'CL02', name:'Robotics Club'}),
(:Club {club_id:'CL03', name:'Music Club'});
```

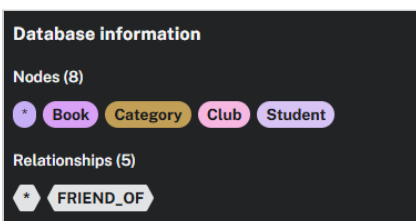


ผลลัพธ์: Nodes (8) คือ Student 5 และ Club 3

2.2 สร้าง Relationship FRIEND_OF

กำหนดความเป็นเพื่อนระหว่างนักเรียน เพื่อใช้ในการ Traversal และ Recommendation

```
MATCH (s1:Student {student_id:'S001'}), (s2:Student {student_id:'S002'}),
(s3:Student {student_id:'S003'}), (s4:Student {student_id:'S004'}),
(s5:Student {student_id:'S005'})
CREATE (s1)-[:FRIEND_OF]->(s2), (s1)-[:FRIEND_OF]->(s3),
(s1)-[:FRIEND_OF]->(s4), (s3)-[:FRIEND_OF]->(s5),
(s4)-[:FRIEND_OF]->(s5);
```



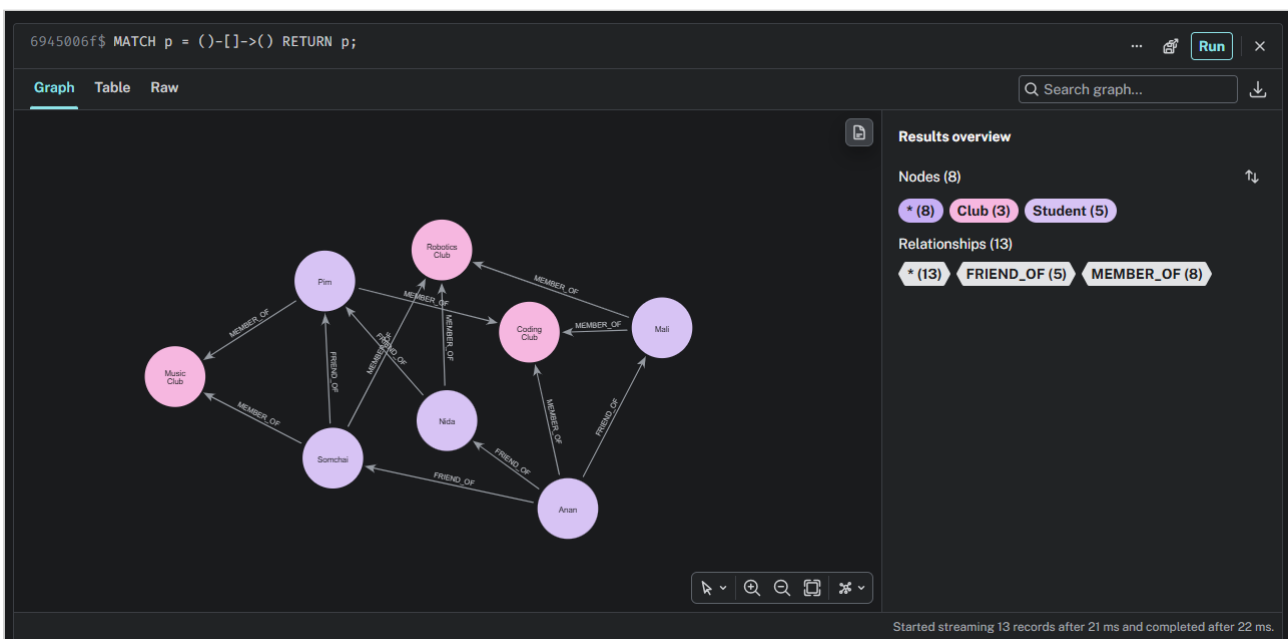
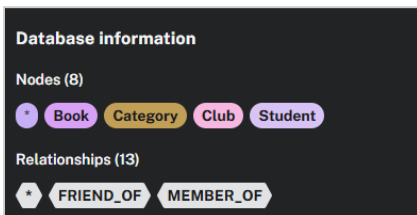
ผลลัพธ์: Relationships (5)

2.3 สร้าง Relationship MEMBER_OF และดูกราฟทั้งหมด

กำหนดว่านักเรียนแต่ละคนเป็นสมาชิกของชมรมใด

```
MATCH (s1:Student {student_id:'S001'}), (s2:Student {student_id:'S002'}),
      (s3:Student {student_id:'S003'}), (s4:Student {student_id:'S004'}),
      (s5:Student {student_id:'S005'}),
      (c1:Club {club_id:'CL01'}), (c2:Club {club_id:'CL02'}),
      (c3:Club {club_id:'CL03'})
CREATE (s1)-[:MEMBER_OF]->(c1), (s2)-[:MEMBER_OF]->(c1),
      (s2)-[:MEMBER_OF]->(c2), (s3)-[:MEMBER_OF]->(c2),
      (s3)-[:MEMBER_OF]->(c3), (s4)-[:MEMBER_OF]->(c2),
      (s5)-[:MEMBER_OF]->(c1), (s5)-[:MEMBER_OF]->(c3);

MATCH p = ()-[]->() RETURN p;
```

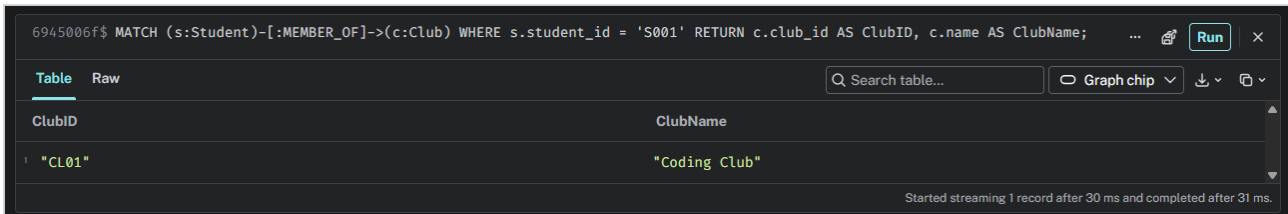


ผลลัพธ์: Relationships (13) คือ FRIEND_OF 5 และ MEMBER_OF 8

2.4 ใช้ MATCH + WHERE + RETURN ดูชมรมของ S001

ค้นหาชมรมที่ S001 เป็นสมาชิกอยู่

```
MATCH (s:Student)-[:MEMBER_OF]->(c:Club)
WHERE s.student_id = 'S001'
RETURN c.club_id AS ClubID, c.name AS ClubName;
```



6945006f\$ MATCH (s:Student)-[:MEMBER_OF]->(c:Club) WHERE s.student_id = 'S001' RETURN c.club_id AS ClubID, c.name AS ClubName; ... Run X

Table	Raw
ClubID	ClubName
1 "CL01"	"Coding Club"

Started streaming 1 record after 30 ms and completed after 31 ms.

ผลลัพธ์: S001 เป็นสมาชิก Coding Club

2.5 ใช้ Traversal ไปยัง Club

เริ่มจาก S001 → FRIEND_OF → เพื่อน → MEMBER_OF → Club เพื่อดูว่าเพื่อนของ S001 อยู่ชมรมใดบ้าง

```
MATCH (s:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
-[:MEMBER_OF]->(club:Club)
RETURN friend.name AS Friend, club.name AS Club
ORDER BY Friend;
```



6945006f\$ MATCH (s:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student) -[:MEMBER_OF]->(club:Club) RETURN friend.name AS Fri ... Run X

Table	Raw
Friend	Club
1 "Mali"	"Coding Club"
2 "Mali"	"Robotics Club"
3 "Nida"	"Robotics Club"
4 "Somchai"	"Robotics Club"
5 "Somchai"	"Music Club"

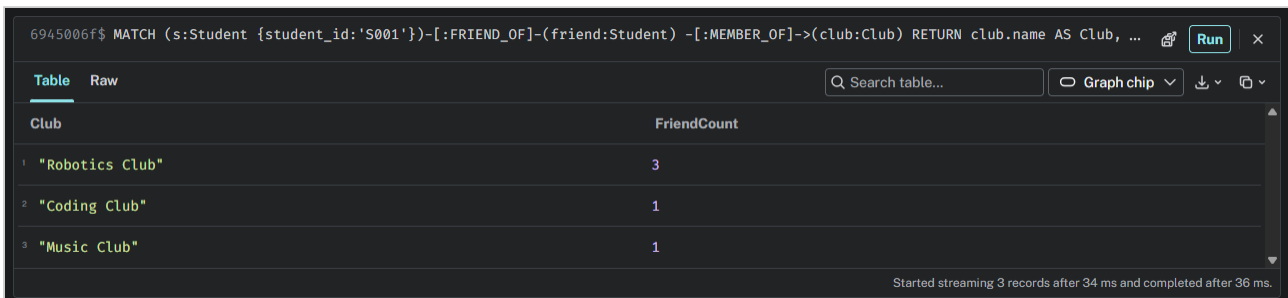
Started streaming 5 records after 45 ms and completed after 46 ms.

ผลลัพธ์: ได้ 5 รายการ จากเพื่อน 3 คน (Mali, Nida, Somchai)

2.6 ใช้ count(DISTINCT friend) ให้คะแนน

นับจำนวนเพื่อนของ S001 ที่อยู่ในแต่ละชมรม แล้วเรียงจากมากไปน้อย

```
MATCH (s:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
      -[:MEMBER_OF]->(club:Club)
RETURN club.name AS Club, count(DISTINCT friend) AS FriendCount
ORDER BY FriendCount DESC;
```



6945006f\$ MATCH (s:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student) -[:MEMBER_OF]->(club:Club) RETURN club.name AS Club, ... Run X

Table	Raw
Club	FriendCount
1 "Robotics Club"	3
2 "Coding Club"	1
3 "Music Club"	1

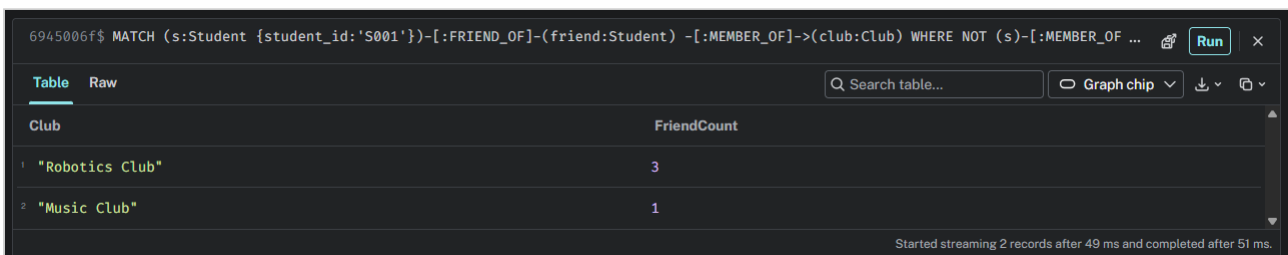
Started streaming 3 records after 34 ms and completed after 36 ms.

ผลลัพธ์: Robotics Club 3 คะแนน, Coding Club 1 คะแนน, Music Club 1 คะแนน

2.7 ตัด Club ที่ S001 เป็น MEMBER_OF อยู่แล้วออก

ใช้ WHERE NOT ตัดชมรมที่ S001 เป็นสมาชิกอยู่แล้ว

```
MATCH (s:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
      -[:MEMBER_OF]->(club:Club)
WHERE NOT (s)-[:MEMBER_OF]->(club)
RETURN club.name AS Club, count(DISTINCT friend) AS FriendCount
ORDER BY FriendCount DESC;
```



6945006f\$ MATCH (s:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student) -[:MEMBER_OF]->(club:Club) WHERE NOT (s)-[:MEMBER_OF] ... Run X

Table	Raw
Club	FriendCount
1 "Robotics Club"	3
2 "Music Club"	1

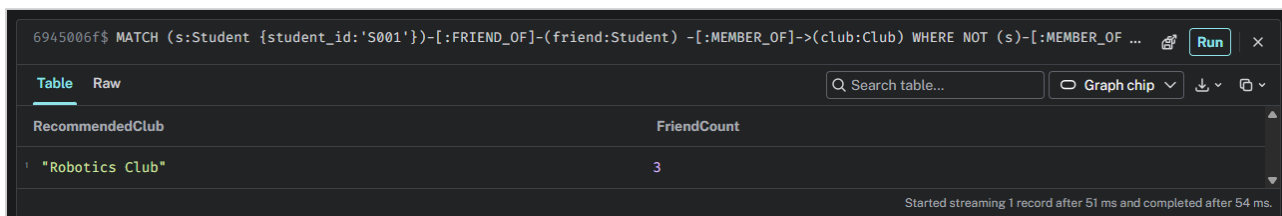
Started streaming 2 records after 49 ms and completed after 51 ms.

ผลลัพธ์: Coding Club ถูกตัดออก เหลือ Robotics Club 3 และ Music Club 1

2.8 สร้าง Recommendation อันดับ 1

เลือกชมรมที่ได้คะแนนสูงสุดด้วย LIMIT 1

```
MATCH (s:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)
      -[:MEMBER_OF]->(club:Club)
WHERE NOT (s)-[:MEMBER_OF]->(club)
RETURN club.name AS RecommendedClub, count(DISTINCT friend) AS FriendCount
ORDER BY FriendCount DESC
LIMIT 1;
```



6945006f\$ MATCH (s:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student) -[:MEMBER_OF]->(club:Club) WHERE NOT (s)-[:MEMBER_OF] ... Run

Table	Raw
RecommendedClub	FriendCount
"Robotics Club"	3

Started streaming 1 record after 51 ms and completed after 54 ms.

ผลลัพธ์: ระบบแนะนำ Robotics Club ให้ S001

ทำไมระบบจึงแนะนำ Robotics Club เป็นอันดับ 1

ระบบเริ่มจากนักเรียน S001 (Anan) แล้วเดินทางผ่านความสัมพันธ์ FRIEND_OF ไปยังเพื่อนทั้ง 3 คน คือ Mali, Somchai และ Nida จากนั้นเดินทางต่อผ่าน MEMBER_OF ไปยังชมรมที่เพื่อนแต่ละคนเป็นสมาชิก และใช้ count(DISTINCT friend) นับว่าแต่ละชมรมมีเพื่อนของ S001 อยู่กี่คน

Robotics Club มีเพื่อนของ S001 เป็นสมาชิกครบทั้ง 3 คน ขณะที่ Coding Club และ Music Club มีเพียงชมรมละ 1 คน เมื่อใช้ WHERE NOT ตัด Coding Club ซึ่ง S001 เป็นสมาชิกอยู่แล้วออก Robotics Club จึงเป็นชมรมที่ได้คะแนนสูงสุดและถูกเลือกเป็น Recommendation อันดับ 1 ด้วย ORDER BY FriendCount DESC LIMIT 1

ส่วน Pim (S005) แม้จะเป็นเพื่อนของ Somchai และ Nida แต่ไม่ได้เป็นเพื่อนโดยตรงของ S001 จึงไม่ถูกนับในการคำนวณ เพราะ query พิจารณาเฉพาะเพื่อนระยะ 1 hop เท่านั้น